Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Battle Code 2

Semester II Tahun 2025/2026

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Tugas **2**
1.1 Latar Belakang . . . 2
1.2 Komponen Permainan . . . 2
1.2.1 Objective Permainan . . . 2
1.2.2 Aturan Tiebreaker . . . 2
1.2.3 Peta Permainan . . . 2
1.2.4 Jarak Pandang dan Pergerakan . . . 2
1.2.5 Tower dan Ruins . . . 3
1.2.6 Sumber Daya . . . 3
1.2.7 Robot . . . 3
1.2.8 Jenis Robot . . . 3
1.2.9 Komunikasi . . . 4
1.2.10 Batasan Program . . . 4
1.2.11 Tujuan Tugas Besar . . . 4

2 Landasan Teori **4**
2.1 Algoritma Greedy . . . 4
2.1.1 Konsep Dasar . . . 4
2.1.2 Elemen-Elemen . . . 5
2.2 Mekanisme Kerja Program dan Implementasi Greedy pada Battlecode . . . 5
2.2.1 Sistem Eksekusi Berbasis Giliran (Turn-Based Execution) . . . 5
2.2.2 Penerapan Algoritma Greedy pada Bot . . . 6
2.2.3 Eksekusi dan Siklus Hidup Bot . . . 6

3 Aplikasi Strategi Greedy **6**
3.1 Pemetaan Persoalan Battlecode ke Elemen Algoritma Greedy . . . 6
3.2 Eksplorasi Alternatif Solusi Greedy . . . 6
3.2.1 Alternatif 1: Greedy Spasial Dinamis Berbobot (Solusi Bot Utama) . . . 6
3.2.2 Alternatif 2: [Juan] . . . 8
3.2.3 Alternatif 3: [Edo] . . . 8
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi . . . 8
3.4 Strategi Greedy yang Dipilih beserta Pertimbangan . . . 8

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode merupakan permainan strategi waktu nyata di mana pemain menulis kode untuk mengendalikan agen otonom berupa robot. Robot-robot tersebut harus bekerja sama untuk mengalahkan tim lawan dengan mengelola sumber daya, melakukan koordinasi, serta menerapkan strategi yang efektif.

Dalam permainan ini, pemain harus memanfaatkan berbagai konsep ilmu komputer seperti kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, serta mekanisme komunikasi antar agen. Selain itu, setiap robot memiliki batasan sumber daya komputasi pada setiap giliran, sehingga pemain perlu merancang algoritma yang efisien dan mampu mengambil keputusan dengan cepat.

Pada Tugas Besar 1 mata kuliah IF2211 Strategi Algoritma ini, mahasiswa diminta untuk merancang dan mengimplementasikan bot permainan Battlecode 2025 yang memanfaatkan strategi algoritma greedy. Bot yang dibuat nantinya akan dipertandingkan dengan bot dari kelompok lain.

1.2 Komponen Permainan

1.2.1 Objective Permainan

Tujuan utama permainan Battlecode adalah mewarnai wilayah peta sebanyak mungkin. Tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai akan memenangkan permainan.

Selain itu, kemenangan juga dapat dicapai dengan menghancurkan seluruh robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Apabila setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% peta, maka permainan akan berakhir dan pemenang ditentukan berdasarkan kriteria berikut secara berurutan:

1. Luas area peta yang telah diwarnai
2. Jumlah menara sekutu yang masih hidup
3. Total jumlah uang (chips)
4. Total jumlah cat (paint) yang dimiliki robot dan menara
5. Jumlah robot sekutu yang masih hidup
6. Jika masih seri, pemenang ditentukan secara acak

1.2.3 Peta Permainan

Setiap permainan dimainkan pada peta berbentuk grid dua dimensi berbentuk persegi panjang dengan ukuran antara 20×20 hingga 60×60 petak.

Koordinat peta dimulai dari titik kiri bawah $(0, 0)$ dan meningkat ke arah timur (kanan) dan utara (atas). Setiap lokasi pada peta direpresentasikan oleh objek `MapLocation` yang menyimpan koordinat x dan y.

Untuk menjaga keseimbangan permainan, peta dijamin memiliki sifat simetris, baik melalui rotasi maupun refleksi.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat lingkungan di sekitarnya dalam radius $\sqrt{20}$ atau sekitar 4.47 petak. Robot tidak dapat bergerak melewati menara maupun robot lain.

Peta juga dapat memiliki tembok (walls) yang tidak dapat dilewati maupun diwarnai. Jumlah tembok maksimal adalah 20% dari seluruh petak pada peta. Selain itu, robot juga tidak dapat melewati atau mewarnai reruntuhan (ruins).

1.2.5 Tower dan Ruins

Menara memiliki fungsi sebagai zona spawn tim sekaligus unit pertahanan yang dapat menyerang musuh. Untuk membangun menara, pemain harus mewarnai pola khusus pada area reruntuhan (ruins) berukuran 5×5.

Setiap permainan dimulai dengan sejumlah reruntuhan yang telah tersedia pada peta. Reruntuhan tidak dapat dipindahkan maupun dihancurkan. Pusat reruntuhan memiliki jarak minimal lima petak satu sama lain, dan area di sekitarnya dijamin bebas dari tembok.

Setiap tim memulai permainan dengan dua menara, yaitu:

- Menara penghasil uang (Money Tower)
- Menara penghasil cat (Paint Tower)

1.2.6 Sumber Daya

Dalam permainan Battlecode terdapat dua sumber daya utama, yaitu **paint** dan **chips**.

- Paint

  Paint merupakan sumber daya utama yang digunakan untuk mewarnai wilayah peta dan menentukan skor tim. Setiap robot dan menara memiliki kapasitas penyimpanan paint yang berbeda.Robot yang memiliki paint rendah akan mengalami peningkatan cooldown aksi, bahkan tidak dapat melakukan aksi apabila paint habis sepenuhnya.

- Chips

  Chips digunakan untuk membangun robot maupun menara. Chips dihasilkan secara pasif oleh menara penghasil uang setiap ronde.Setiap tim memulai permainan dengan 2500 chips.

1.2.7 Robot

Robot merupakan unit utama dalam permainan. Setiap robot dapat melakukan berbagai aksi seperti bergerak, menyerang, melakukan sensing, serta berkomunikasi dengan robot lain.

Setiap robot memiliki batas komputasi yang disebut *bytecode*. Jika robot melebihi batas bytecode pada satu giliran, eksekusi program akan dihentikan sementara dan dilanjutkan pada ronde berikutnya.

Robot memiliki atribut utama berupa:

- Hitpoints (HP)
- Kapasitas paint
- Cooldown aksi dan pergerakan
- Kemampuan khusus sesuai tipe robot

1.2.8 Jenis Robot

Permainan Battlecode menyediakan beberapa jenis robot dengan karakteristik berbeda.

- Soldier

  Robot tipe Soldier memiliki kemampuan menyerang satu petak dan dapat memberikan damage terhadap menara musuh.

- Splasher

  Robot tipe Splasher memiliki serangan area (area of effect) yang mampu mewarnai atau menimpa cat pada beberapa petak sekaligus.

- Mopper

  Robot tipe Mopper memiliki kemampuan untuk menghapus cat milik musuh serta mentransfer paint kepada robot atau menara sekutu.

1.2.9 Komunikasi

Robot tidak dapat berbagi variabel secara langsung karena setiap robot menjalankan salinan program yang terpisah. Oleh karena itu, komunikasi dilakukan melalui dua mekanisme utama:

- **Pesan (messages)** yang dapat dikirim antar robot atau menara
- **Penanda (markers)** yang ditempatkan pada peta untuk memberikan informasi kepada robot sekutu

Setiap pesan memiliki ukuran maksimal 4 byte dan hanya disimpan selama lima ronde dalam buffer penerima.

1.2.10 Batasan Program

Program robot memiliki beberapa batasan yang harus dipatuhi.

- Bytecode

  Setiap robot memiliki batas komputasi sebesar 17500 bytecode per ronde, sedangkan menara memiliki batas 20000 bytecode per ronde.

- Memori

  Penggunaan memori robot tidak boleh melebihi 8 MB selama pertandingan.

- Execution Time

  Jika total waktu eksekusi sebuah tim melebihi batas yang ditentukan oleh sistem, maka tim tersebut akan otomatis kalah.

1.2.11 Tujuan Tugas Besar

Pada tugas besar ini, setiap kelompok diminta untuk:

- Mengembangkan tiga bot Battlecode menggunakan bahasa Java
- Mengimplementasikan strategi algoritma greedy pada masing-masing bot
- Menggunakan heuristic yang berbeda untuk setiap strategi greedy
- Mengoptimalkan strategi agar mampu memenangkan pertandingan

Bot yang dikembangkan akan dipertandingkan dengan bot dari kelompok lain dalam kompetisi yang diselenggarakan pada akhir tugas besar.

2 Landasan Teori

2.1 Algoritma Greedy

2.1.1 Konsep Dasar

Algoritma *greedy* merupakan suatu metode sederhana yang paling populer untuk memecahkan persoalan optimasi (*optimization problems*), yaitu persoalan mencari solusi optimal. Terdapat dua jenis persoalan optimasi:

- Maksimasi (*maximization*) - contohnya adalah profit, reward, dan lain sebagainya.
- Minimasi (*minimization*) - contohnya adalah *cost*, waktu, jarak, dan lain sebagainya.

Algoritma ini menyarankan pembentukan sebuah solusi secara bertahap, di mana setiap langkah memiliki banyak pilihan yang bisa dievaluasi hingga akhirnya solusi lengkap untuk masalah tersebut bisa tercapai. Penentuan solusi yang bisa dievaluasi pada setiap tahap harus memenuhi kriteria-kriteria berikut:

- Pilihan tidak boleh melanggar *constraints* dari permasalah yang sedang diselesaikan
- Pilihan yang diambil pada tiap langkahnya haruslah merupakan pilihan yang optimal secara lokal (pilihan terbaik yang tersedia pada tahap tersebut)

- Pilihan yang sudah dibuat bersifat final dan tidak dapat diubah maupun ditarik kembali pada langkah-langkah berikutnya dalam algoritma.

Tiga karakteristik barusan menjelaskan mengapa algoritma ini disebut sebagai algoritma *greedy*. Setiap langkah yang diambil harus memenuhi prinsip "*take what you can get now!*" tanpa perlu memikirkan akibat yang dihasilkan di langkah-langkah berikutnya. Setiap langkah optimum lokal yang diambil diharapkan bisa menghasilkan solusi keseluruhan yang optimum secara global.

Algoritma ini secara spesifik hanya bisa diaplikasikan untuk tipe permasalahan optimasi. Tipe persoalan optimasi adalah persoalan yang memiliki tujuan untuk mencari solusi optimal dari berbagai kemungkinan solusi yang ada. Secara umum, permalasahan optimasi dibagi menjadi dua jenis, maksimasi (*maximization*), yaitu persoalan optimasi yang bertujuan untuk mendapatkan hasil maksimal dari fungsi tujuan, dan minimasi (*minimization*), yaitu persoalan optimasi yang bertujuan untuk mendapatkan hasil minimal dari fungsi tujuan.

Walaupun setiap langkah yang diambil merupakan optimum lokal dari semua pilihan yang tersedia, perlu diingat bahwa solusi yang dihasilkan belum tentu merupakan sebuah optimum global. Pendekatan menggunakan algoritma ini bisa dijadikan sebagai solusi hampiran, yang pada beberapa jenis persoalan tertentu sudah dianggap cukup dan dapat diterima, mengingat efisiensi waktu yang jauh lebih cepat dibanding dengan algoritma *exhaustive search*.

2.1.2 Elemen-Elemen

Dalam memecahkan tipe permasalahan optimasi, algoritma *greedy* melibatkan pencarian sebuah himpunan bagian dari sekumpulan kandidat yang ada. Himpunan bagian tersebut harus memenuhi beberapa kriteria tertentu agar bisa merepresentasikan suatu solusi yang dioptimalkan oleh sebuah fungsi tujuan. Untuk mencapai hal terrsebut, algoritma *greedy* biasanya dibangun oleh beberapa elemen pendukung:

1. **Himpunan Kandidat (*Candidate set*)**
   Ini adalah sekumpulan elemen ytang berisi seluruh kandidat pilihan yang bisa dipilih pada setiap langkah pencarian. Sebagai contoh, dalam kasus komputasi, kandidat dapat beruapa simpul ataupun sisi di dalam graf, pecahan mata uang, kumpulan benda berbobot tertentu, dan hal-hal lain yang relevan dengan permasalahan yang sedang dioptimasi.

2. **Himpunan Solusi (*Solution set*)**
   Himpunan solusi adalah sebuah himpunan yang berisi solusi-solusi yang dipilih pada setiap langkah yang sudah diambil. Himpunan ini dibangun secara bertahap dengan kondisi awal kosong dan akan terus ditambahkan elemen baru berupa solusi yang dipilih pada setiap iterasi pemilihan langkah hingga akhir solusi akhir tercapai.

3. **Fungsi Seleksi (*Selection function*)**
   Fungsi ini bertugas untuk memiliki kandidat terbaik dari himpunan kandidat pada suatu langkah. Fungsi ini didasarkan pada strategi *greedy* (heuristik) tertentu yang memberikan kondisi lokal optimum pada langkah tersebut.

4. **Fungsi Kelayakan (*Feasible function*)**
   Fungsi ini bertugas untuk menguji output fungsi seleksi untuk menentukan layak atau tidaknya hasil tersebut dimasukkan ke himpunan solusi. Pengcekan ini penting untuk memastikan bahwa output dari fungsi seleksi tidak melanggar *constraint* yang ditetapkan oleh permasalah.

5. **Fungsi Objektif (*Objective function*)**
   Fungsi ini bertugas untuk mengukur ketercapaian keseluruhan ataupun nilai dari sebuah solusi yang dihasilkan. Inilah fungsi utama yang bertugas untuk mencapai target akhir persoalan, yaitu memaksimalisasi atau meminimalisasi nilai dari solusi yang didapatkan.

2.2 Mekanisme Kerja Program dan Implementasi Greedy pada Battlecode

2.2.1 Sistem Eksekusi Berbasis Giliran (Turn-Based Execution)

Dalam permainan Battlecode, setiap robot menjalankan siklus hidupnya melalui struktur *infinite loop* yang mengeksekusi instruksi pada setiap ronde (*turn*). Karena adanya limitasi *bytecode* (17.500 untuk robot, 20.000 untuk menara), robot tidak dapat melakukan komputasi yang berat seperti algoritma penelusuran jalur (*pathfinding*) global secara menyeluruh pada setiap giliran. Oleh karena itu, program menggunakan pola *Finite State Machine* (FSM) di mana robot akan mengevaluasi kondisi sekitarnya dan memutuskan transisi *state* (misalnya: `EXPLORE`, `PAINT`, `BUILD_TOWER`).

2.2.2 Penerapan Algoritma Greedy pada Bot

Algoritma *greedy* diimplementasikan sebagai inti dari pengambilan keputusan robot, baik untuk pergerakan (*movement*) maupun aksi (*action*). Implementasi ini dilakukan melalui pendekatan heuristik spasial berbobot (*weighted spatial heuristic*).

Alih-alih merencanakan serangkaian langkah panjang yang menguras *bytecode*, robot akan mengevaluasi petak-petak (*tiles*) yang berada tepat di sekitarnya. Robot memberikan skor pada setiap petak berdasarkan parameter taktis (seperti keberadaan cat musuh, reruntuhan, atau teman yang berdempetan). Keputusan *greedy* diambil dengan memilih petak yang memberikan skor heuristik tertinggi (optimum lokal) pada giliran tersebut.

2.2.3 Eksekusi dan Siklus Hidup Bot

Secara arsitektur, program dijalankan melalui sebuah kelas utama `RobotPlayer` yang bertindak sebagai titik masuk (*entry point*). Berdasarkan `UnitType`, kontrol dilempar ke kelas spesifik pewaris `BaseRobot` (seperti `SoldierUnit`, `SplasherUnit`, dll.). Setiap giliran, agen robot akan memanggil fungsi `tick()`, membaca pesan dari *broadcasting* menara, menjalankan fungsi *anti-clustering* (penghindar dempet), dan mengeksekusi aksi *greedy*-nya sebelum akhirnya memanggil `Clock.yield()` untuk menyerahkan sisa eksekusi kepada *engine* permainan.

3 Aplikasi Strategi Greedy

3.1 Pemetaan Persoalan Battlecode ke Elemen Algoritma Greedy

Persoalan dominasi wilayah dan manajemen agen otonom dalam Battlecode 2025 merupakan persoalan optimasi dinamis dengan batasan sumber daya komputasi (*bytecode*). Berdasarkan landasan teori optimasi, persoalan ini dapat dipetakan ke dalam lima elemen dasar algoritma *Greedy* sebagai berikut:

1. **Himpunan Kandidat (*Candidate Set*):** Kumpulan arah pergerakan yang terdiri dari 8 arah mata angin di sekitar posisi robot saat ini, ditambah dengan serangkaian aksi spasial (menyerang petak, mewarnai pola, atau mentransfer sumber daya) yang berada dalam radius aksi agen tersebut.

2. **Himpunan Solusi (*Solution Set*):** Serangkaian keputusan pergerakan (*movement path*) dan tindakan (*action sequence*) yang valid, yang pada akhirnya mendominasi wilayah peta serta mendirikan infrastruktur menara sekutu.

3. **Fungsi Kelayakan (*Feasible Function*):** Modul validasi yang memastikan kandidat langkah tidak melanggar aturan permainan. Kondisi ini diperiksa melalui *API* permainan, seperti pengecekan rintangan dinding (`isWall()`), keberadaan unit lain yang menghalangi jalan, serta ketersediaan sumber daya *paint* sebelum mengeksekusi suatu aksi.

4. **Fungsi Objektif (*Objective Function*):** Fungsi yang bertujuan untuk memaksimalkan jumlah petak yang diwarnai oleh cat sekutu, memaksimalkan rasio kerusakan terhadap menara musuh, serta meminimalisir penggunaan *bytecode* pada setiap giliran.

5. **Fungsi Seleksi (*Selection Function*):** Fungsi heuristik berbasis spasial yang mengevaluasi nilai taktis dari setiap kandidat pada Himpunan Kandidat. Fungsi ini akan memilih satu kandidat dengan skor tertinggi (optimum lokal) untuk dimasukkan ke dalam Himpunan Solusi pada giliran tersebut.

3.2 Eksplorasi Alternatif Solusi Greedy

Guna mencari strategi heuristik terbaik yang memenuhi Fungsi Objektif, dieksplorasi tiga pendekatan alternatif solusi *Greedy* yang memiliki karakteristik penentuan optimum lokal yang berbeda.

3.2.1 Alternatif 1: Greedy Spasial Dinamis Berbobot (Solusi Bot Utama)

Alternatif ini menggunakan arsitektur *Finite State Machine* (FSM) untuk menentukan konteks makro agen, yang kemudian dikombinasikan dengan fungsi seleksi *Greedy* berbasis pembobotan (*weighted scoring*) pada eksekusi mikronya. Berikut adalah elaborasi heuristik *Greedy* pada setiap tipe agen otonom:

A. Strategi *Spawn* pada Menara (*TowerUnit*)

Menara mengimplementasikan *Greedy Resource Management* untuk mengubah kepingan uang (*chips*) menjadi unit militer secara maksimal tanpa melakukan penimbunan yang tidak efisien.

- ***Greedy Initial Expansion*:** Pada 30 ronde pertama, sumber daya secara rakus digunakan untuk mencetak dua agen *Soldier*. Agen pertama dipaksa lahir berorientasi ke Barat, dan agen kedua ke Timur untuk mencegah tumpang-tindih area kerja pada fase awal permainan.
- ***Dynamic Thresholding*:** Transisi fase permainan memengaruhi ambang batas (*threshold*) pencetakan unit. Pada fase *late-game*, ketika *chips* melebihi 800, menara secara rakus langsung memanggil *Splasher* (unit termahal) tanpa menunggu perintah.
- ***Threat-Driven Ratio*:** Secara standar, menara menyeimbangkan rasio (*Soldier* 40%, *Splasher* 45%, *Mopper* 15%). Namun, jika radar mendeteksi sinyal keberadaan musuh (`enemyTowerCount > 0`), menara secara *greedy* menunda antrean *Soldier* dan memprioritaskan pencetakan *Mopper* sebagai unit defensif.

B. Unit *Soldier*

Unit ini mengeksekusi ekspansi wilayah dengan fungsi skoring yang menghitung nilai setiap petak di sekitarnya.

- ***Greedy Initial Paint*:** *Soldier* yang berstatus `INITIAL_PAINT` memindai wilayah 5×5 petak di sekitar menaranya. Mereka membagi wilayah evaluasi secara rakus berdasarkan sumbu x (kiri dan kanan), menimpa semua petak netral hingga perimeter menara sekutu sepenuhnya dikuasai.
- ***Ruin Proximity Selection*:** Dalam mencari tapak reruntuhan (*Ruin*) untuk mendirikan menara, agen mengevaluasi semua reruntuhan terdekat menggunakan heuristik:

$$\text{Skor} = (\text{Pola Benar} \times 10) - (\text{Jarak}/5)$$

  Agen akan bergerak ke reruntuhan dengan skor optimum lokal tertinggi.
- ***Paint Routing Heuristic*:** Pergerakan *Soldier* dipandu oleh pembobotan petak. Petak cat musuh diberikan skor dominan (+60), petak netral (+30), sedangkan petak berpelumas cat sekutu diberi penalti (-15) agar unit tidak menetap di wilayah sendiri. Tambahan bobot (+80) diberikan jika arah tersebut memiliki kedekatan dengan reruntuhan.

C. Unit *Mopper*

Sebagai unit pertahanan, fungsi *Greedy* pada *Mopper* diarahkan untuk netralisasi ancaman dan perbaikan struktural.

- ***Greedy Melee Combat*:** Agen menyimulasikan tebasan sabit (`mopSwing`) ke 8 arah mata angin. Arah yang mengenai jumlah unit musuh terbanyak secara matematis dipilih sebagai solusi lokal.
- ***Defensive Clean Targeting*:** Agen memilih petak cat musuh yang akan dibersihkan dengan skoring yang berfokus pada pertahanan markas:

$$\text{Skor} = (100 - \text{Jarak Menara Sekutu}) + (\text{Kepadatan Cat Musuh} \times 5)$$

  Hal ini mengamankan infrastruktur vital dari sabotase area musuh.
- ***Greedy Support*:** Jika agen mendeteksi sekutu dengan sisa persediaan cat kritis ($< 20\%$), *Mopper* akan langsung memotong antrean aksinya untuk mendonasikan cat (*transfer paint*), memastikan sekutu di garis depan tidak mati akibat penalti degradasi energi.

D. Unit *Splasher*

Pengambilan keputusan *Greedy* pada unit ini berfokus pada kerusakan massal dan identifikasi kepadatan musuh.

- ***Splash Area Scoring*:** Setiap proyektil tembakan dievaluasi dampaknya. Percikan ke petak musuh bernilai +3 dan petak kosong bernilai +1. Tembakan hanya akan dilepaskan pada titik pusat yang akumulasi skor percikannya tertinggi.
- ***Quadrant Density Search*:** Saat kehilangan target visual, *Splasher* membelah peta internalnya menjadi 4 kuadran. Agen menghitung rasio cat musuh terhadap petak valid di tiap kuadran, kemudian menetapkan jalur sepanjang 8 langkah menuju kuadran dengan kepadatan musuh (*density*) tertinggi.

E. Mekanisme Pendukung Eksekusi *Greedy*

Algoritma *Greedy* memiliki kelemahan bawaan pada jebakan spasial minimum lokal, sehingga agen dilengkapi dengan fungsi pendukung berikut:

- ***Anti-Stuck Memory* (*Trail Penalty*):** Kelemahan mendasar dari algoritma *Greedy* adalah risiko terjebak pada geometri ruang konkaf (misal: tembok berbentuk "U"). Hal ini dimitigasi dengan menggunakan struktur *array* sirkular $O(1)$ berukuran 50 yang merekam rekam jejak koordinat. Jika fungsi *Greedy* mengevaluasi petak yang telah dilewati, petak tersebut menerima penalti ekstrem (-800), secara paksa merusak optimum lokal lama dan memaksa agen menemukan rute baru.
- ***Anti-Clustering Repulsion*:** Jika agen mengevaluasi bahwa ia berada pada tingkat kepadatan sekutu yang membahayakan (memicu *Clustering Penalty* dari aturan sistem), agen secara rakus menghentikan tugasnya dan mundur sejauh 1-2 langkah menjauhi *Center of Mass* kerumunan.

3.2.2 Alternatif 2: [Juan]

3.2.3 Alternatif 3: [Edo]

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

3.4 Strategi Greedy yang Dipilih beserta Pertimbangan